Knowledge Graph Application
Technical Documentation - Developer Guide

Name of Student	BITS ID	Contribution
K ROMA PAI	2024aa05965@wilp.bits-pilani.ac.in	100%
JITESH GUPTA	2024ab05020@wilp.bits-pilani.ac.in	100%
KOTHA AMITABH	2024ab05195@wilp.bits-pilani.ac.in	100%
JOHIT GARG	2024aa05907@wilp.bits-pilani.ac.in	100%
KARTHIK REDDY S	2024ab05330@wilp.bits-pilani.ac.in	100%

1. Executive Summary

This document provides a comprehensive technical overview of the Knowledge Graph Application, a full-stack web tool designed to visualize relationships between entities (such as transportation networks or social graphs). It details the architectural decisions, code structure, implementation challenges, and setup instructions.

The application leverages a hybrid architecture combining a **Flask (Python)** backend for graph data management and algorithms with a **React (JavaScript)** frontend for interactive, force-directed visualizations.

2. System Architecture

The application follows a **Client-Server Architecture** where the backend serves both the API and the static frontend assets, simplifying deployment into a single unit.

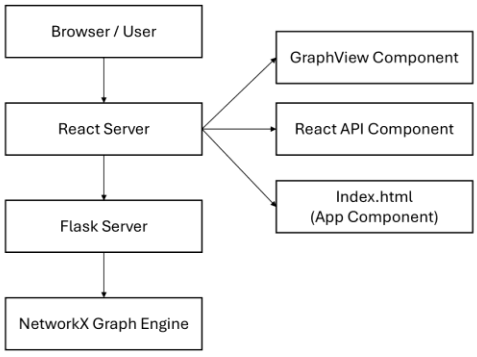

Figure 1 High-Level Diagram

Core Components

1. **Frontend (React 18):**
    - **Visualization Engine:** react-force-graph-2d (Canvas-based rendering).
    - **State Management:** React useState / useEffect hooks.
    - **Communication:** Axios for REST API consumption.
    - **Styling:** CSS Modules with a focus on a responsive Dashboard layout.
2. **Backend (Flask):**
    - **Graph Data Structure:** NetworkX (nx.Graph).
    - **Data Processing:** Pandas for CSV parsing and cleaning.
    - **API Layer:** Flask routes to expose graph operations (CRUD).

3. Backend Development (Flask & NetworkX)

Design Choices

- **Flask:** Chosen for its lightweight nature and simplicity in setting up RESTful endpoints. It allows for rapid iteration compared to Django.
- **NetworkX:** Selected as the graph engine because of its rich library of graph algorithms (shortest path, neighbor finding, centrality).
    - *Trade-off:* It stores the graph in memory (RAM). This is incredibly fast for algorithms but limits the graph size to the server's available memory.**Code Structure: app.py**

The app.py file acts as the central controller. It initializes the G graph object, defines the API routes, and serves the frontend.

Key Logic Blocks:

- **Graph Initialization:** Creates an empty nx.Graph() on startup.
- **Visualization Endpoint (/api/graph_data):** Returns the graph structure formatted as { "nodes": [], "links": [] }. *Crucially, it implements a limit parameter to prevent browser crashes with large datasets.*
- **Search Logic (/api/search):** Performs a fuzzy search on node IDs and expands the result to include neighbors (induced subgraph).

Challenges Faced: Backend

1. **In-Memory Volatility:** Since NetworkX is in-memory, restarting the server wipes the graph. *Solution:* Implemented CSV upload to quickly restore state.
2. **Serialization:** NetworkX objects cannot be directly sent as JSON. *Solution:* We have to manually iterate over G.nodes(data=True) and G.edges(data=True) to convert them into a dictionary format compatible with the frontend.
3. **Directed vs. Undirected:** Handling neighbor expansion for directed graphs requires checking both successors and predecessors, whereas undirected only requires neighbors.

4. Frontend Development (React)

Design Choices

- **React Force Graph:** Chosen over D3.js directly because it offers a React-friendly wrapper around the complex physics simulation, drastically reducing boilerplate code.
- **Canvas Rendering:** We used the <ForceGraph2D> (Canvas) instead of SVG. Canvas is significantly more performant for large graphs (1000+ nodes) as it reduces DOM manipulation overhead.
- **Dashboard Layout:** A persistent sidebar layout was chosen to allow users to interact with controls (Search, Upload) without obstructing the view of the visualization.

Code Structure

- **App.jsx:** The main controller. It holds the graphData state and manages the layout (Sidebar + Main Content). It defines the refreshGraph function which syncs the frontend state with the backend.
- **GraphView.jsx:** A wrapper around react-force-graph. It handles:
    - Custom Node Rendering (Rounded Rectangles).
    - Zoom/Pan interactions.
    - Sparsity settings (Charge/Link Distance).
- **QueryPanel.jsx:** Handles the search input and passes results back to App.jsx.

Challenges Faced: Frontend

1. **The "Hairball" Effect:** With 10,000+ edges, the graph became a solid block of color.
    - *Solution:* Implemented **Sparsity Control** in GraphView (increasing negative charge strength) and a **Visualization Limit** in the backend API to render only a subset by default.
2. **Event Handling Conflict:** The "Zoom on Scroll" behavior of the graph library conflicted with the page scroll.
    - *Solution:* Implemented a non-passive event listener on the canvas DOM element to strictly prevent default scroll behavior within the graph container.
3. **Custom Node Styling:** The default circle nodes were not informative enough.
    - *Solution:* Used the nodeCanvasObject prop to draw custom rounded rectangles with text labels directly on the HTML5 Canvas.

5. Setup & Running Instructions

Prerequisites

- **Python 3.8+** installed.
- **Node.js 16+** installed (for building the frontend).

Step 1: Frontend Build

The React application must be compiled into static files so Flask can serve them.

cd frontend

npm install

npm run build

This creates a build folder inside frontend.

Step 2: Backend Setup

Install the required Python packages.

cd backend

pip install flask flask-cors pandas networkx

Step 3: Running the Application

Start the Flask server. It will serve the API on port 5000 and the React frontend on the root URL.

python app.py

- **Access the App:** Open your browser to http://localhost:5000.
- **API Logs:** Check the server.log file in the backend folder for debug information.

6. Screenshots

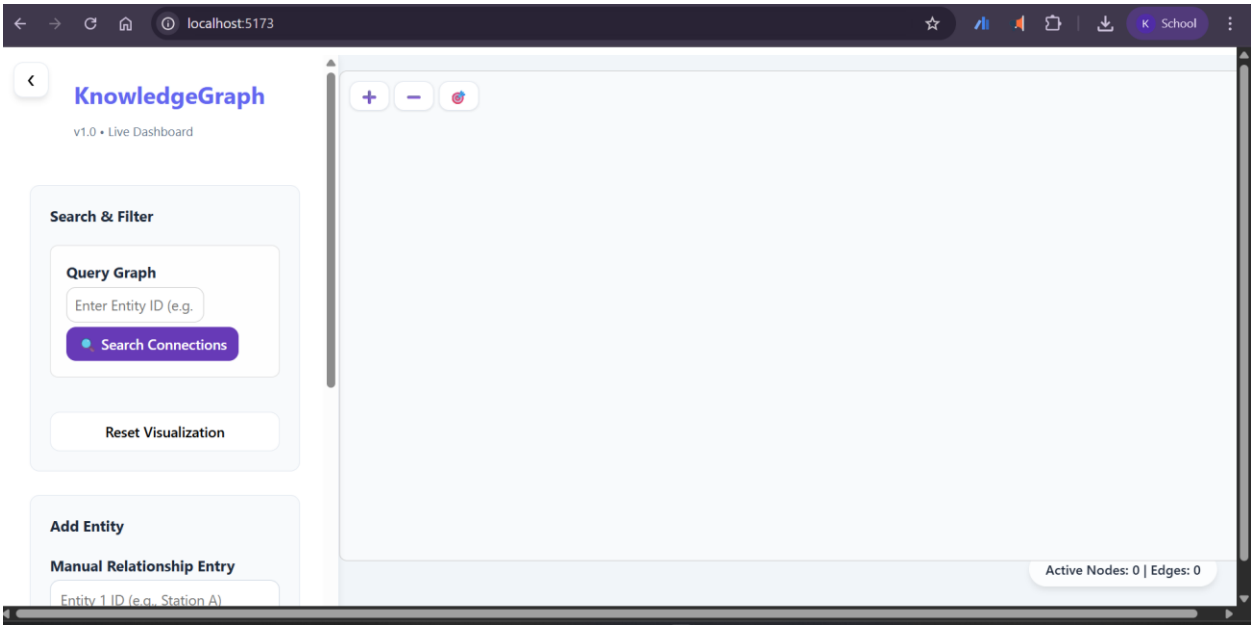

Figure 2 Knowledge Graph Main Dashboard

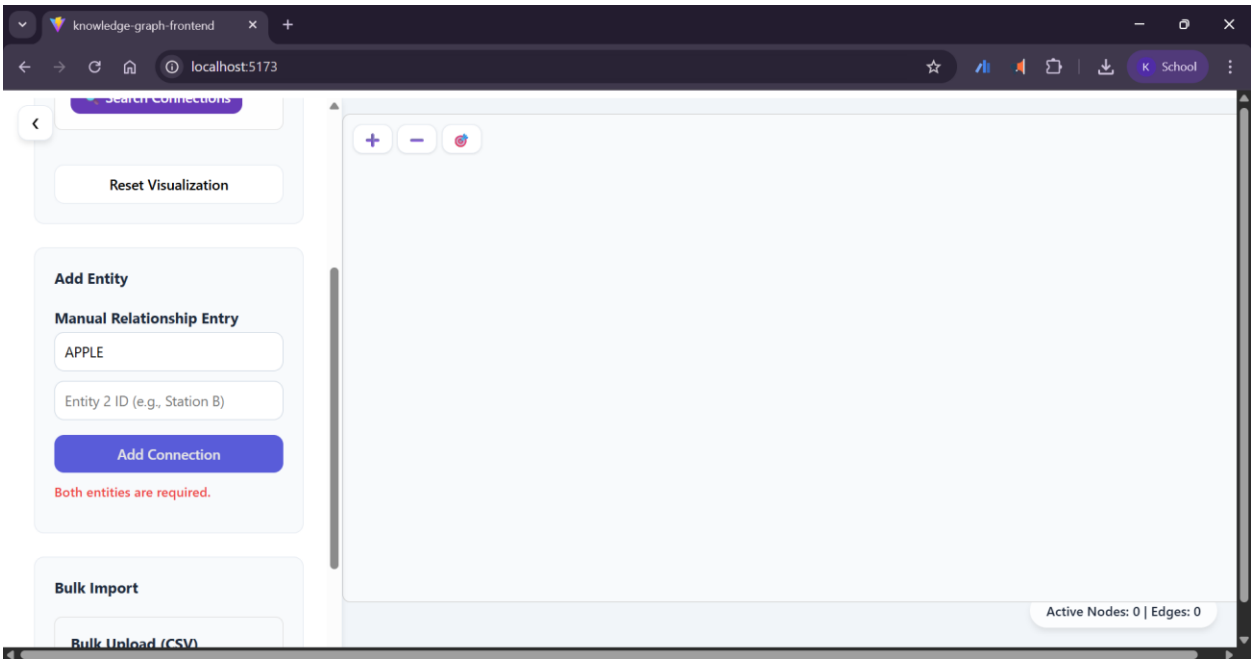

Figure 3 Manual Entry Invalid Failure Scenario

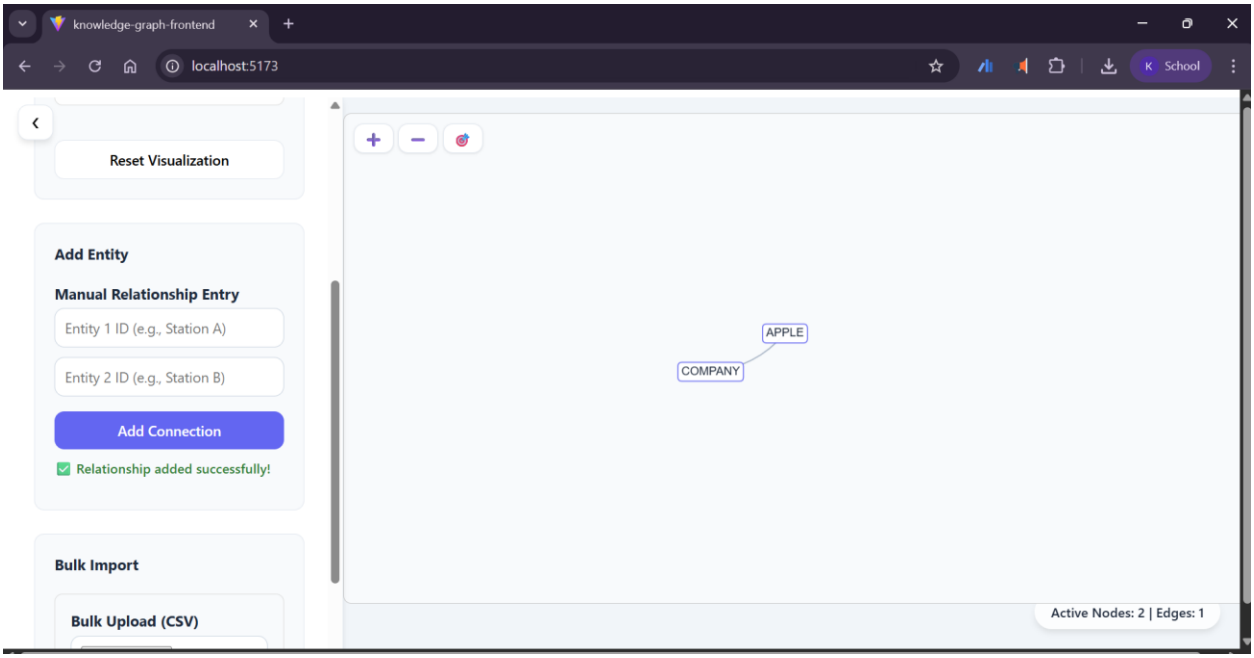

Figure 4 Manual Entry - Success Scenario

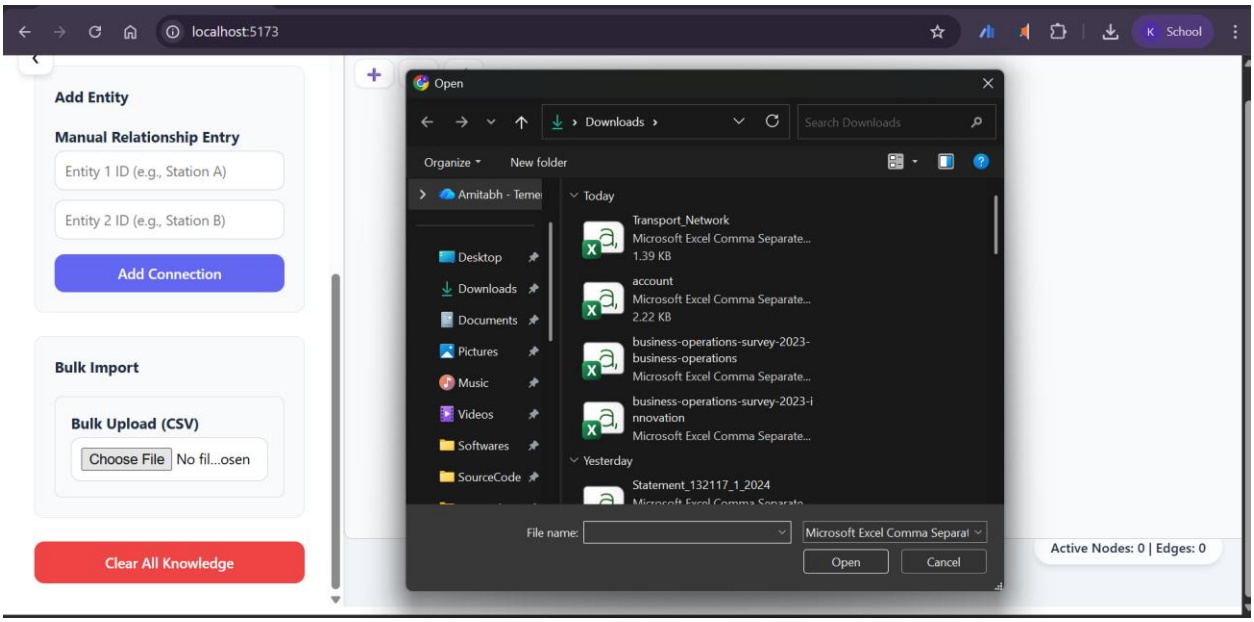

Figure 5 Bulk Upload - CSV – Upload

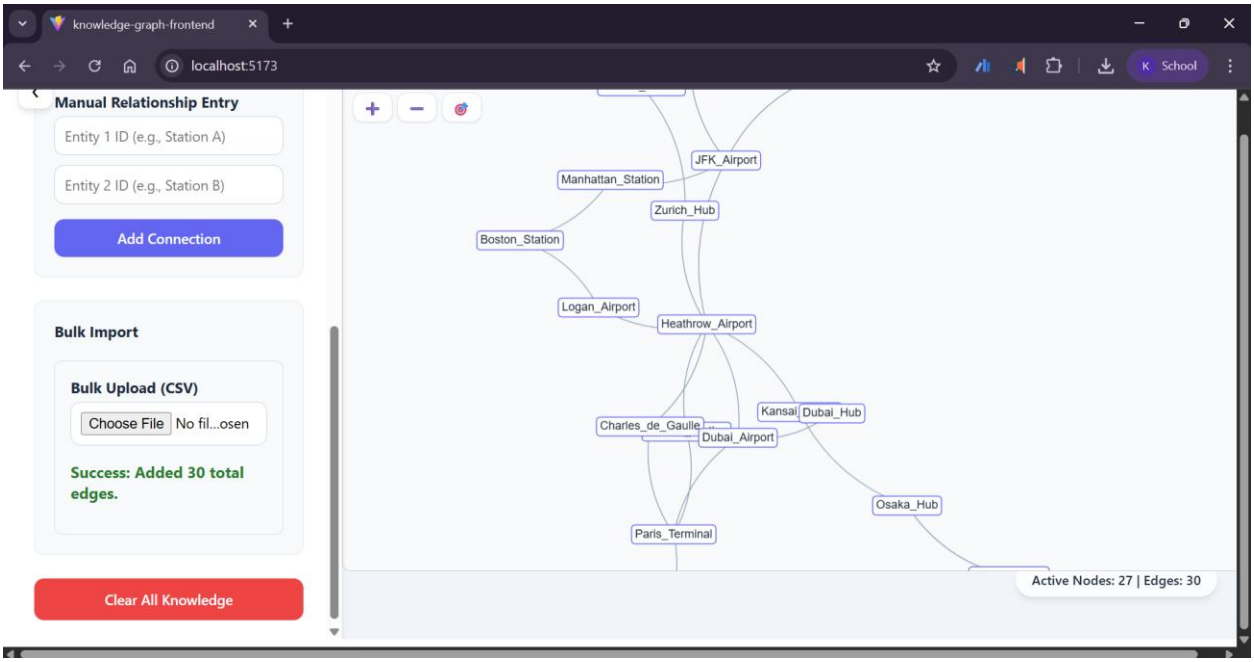

Figure 6 Bulk Upload CSV – Success

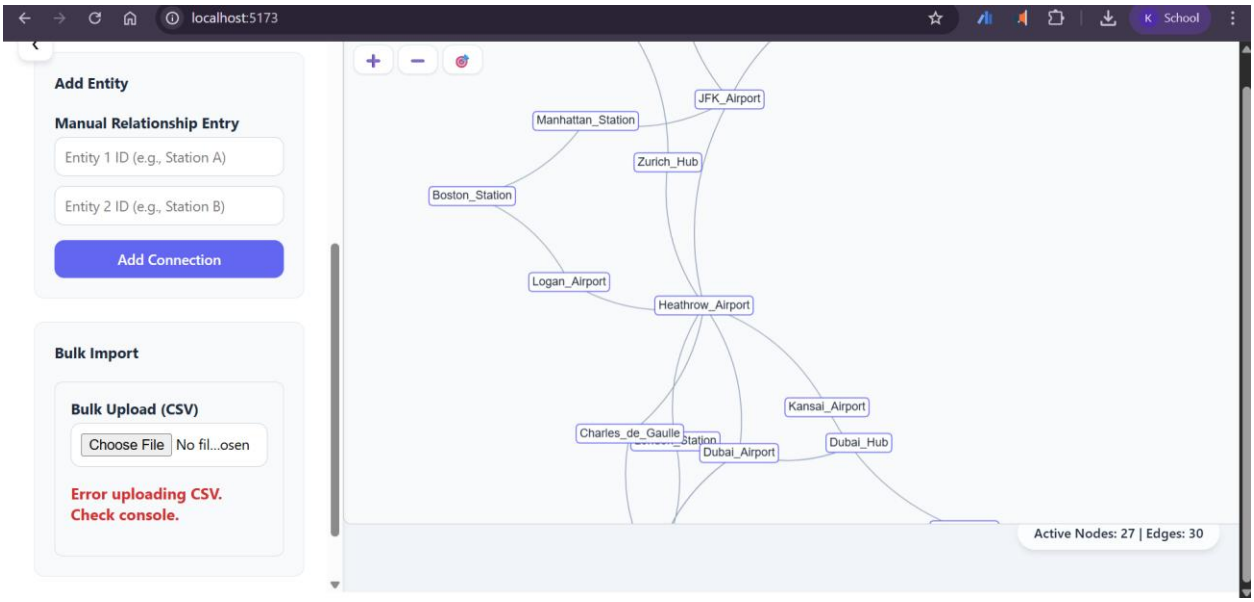

Figure 7 Bulk Upload CSV - Failure

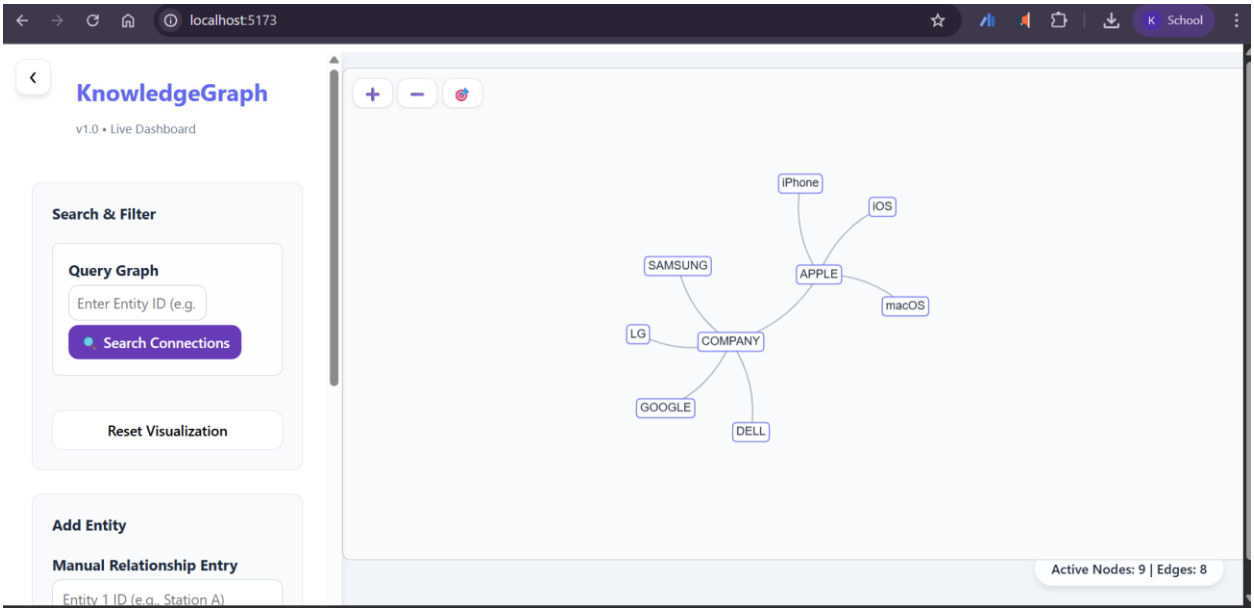

Figure 8 Manual Entry Multiple Node - Query 1

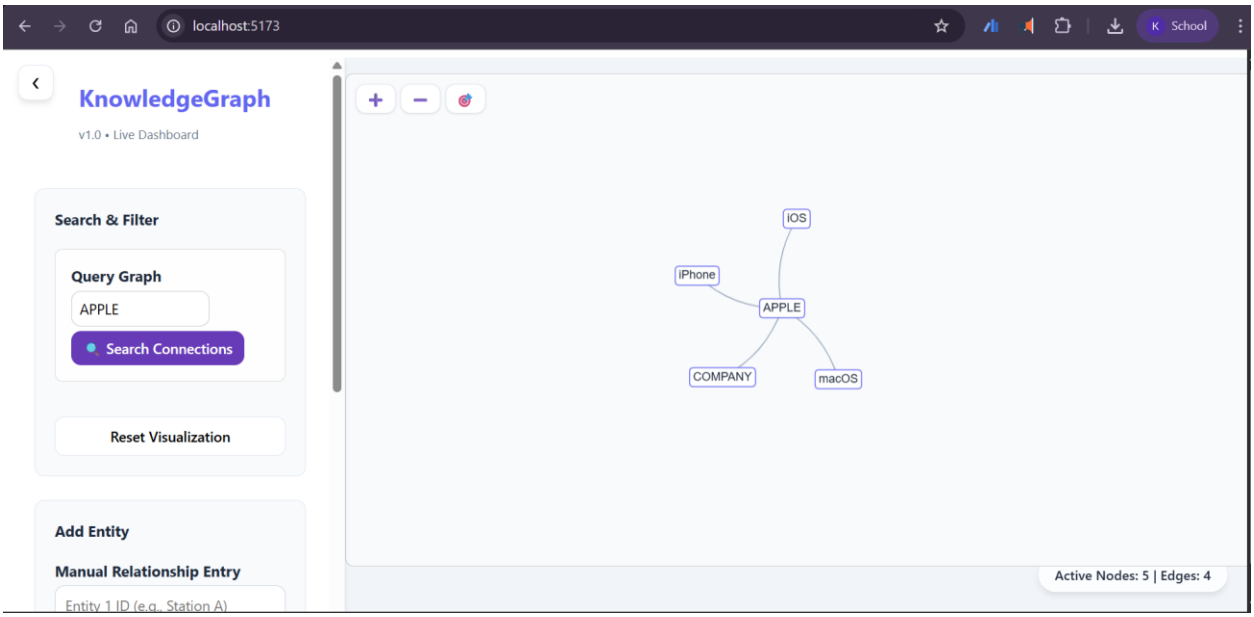

Figure 9 Manual Entry Multiple Node - Query 2

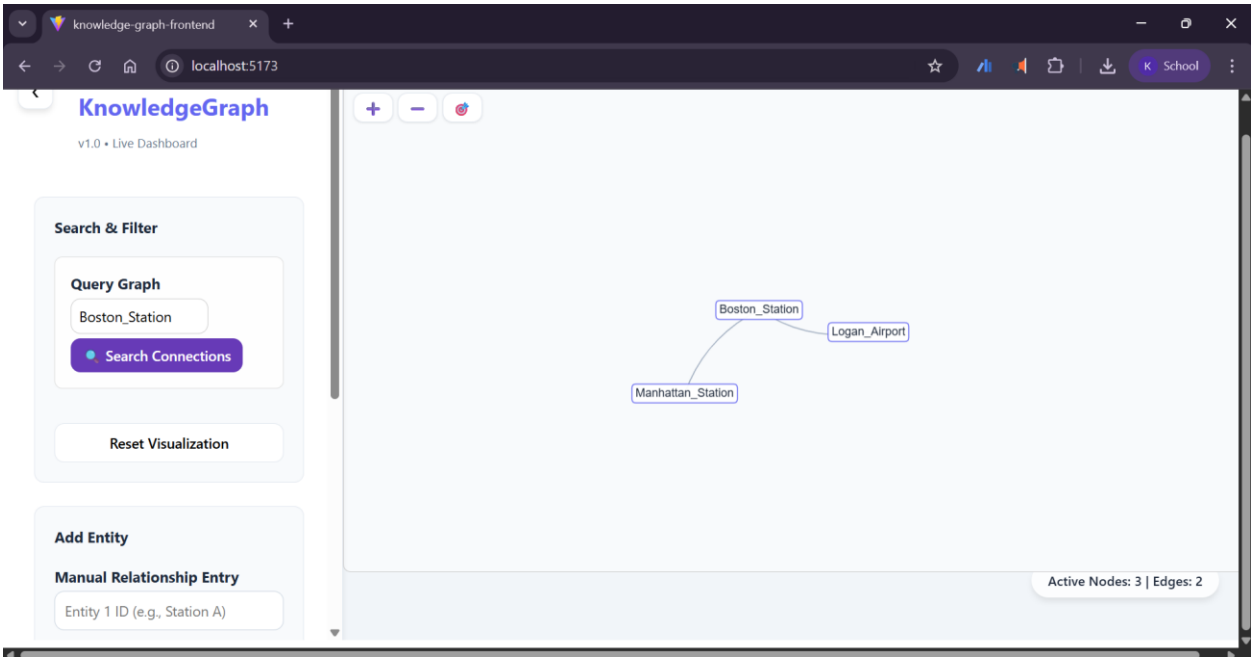

Figure 10 Bulk Upload - Transportation Use Case - Query – Success

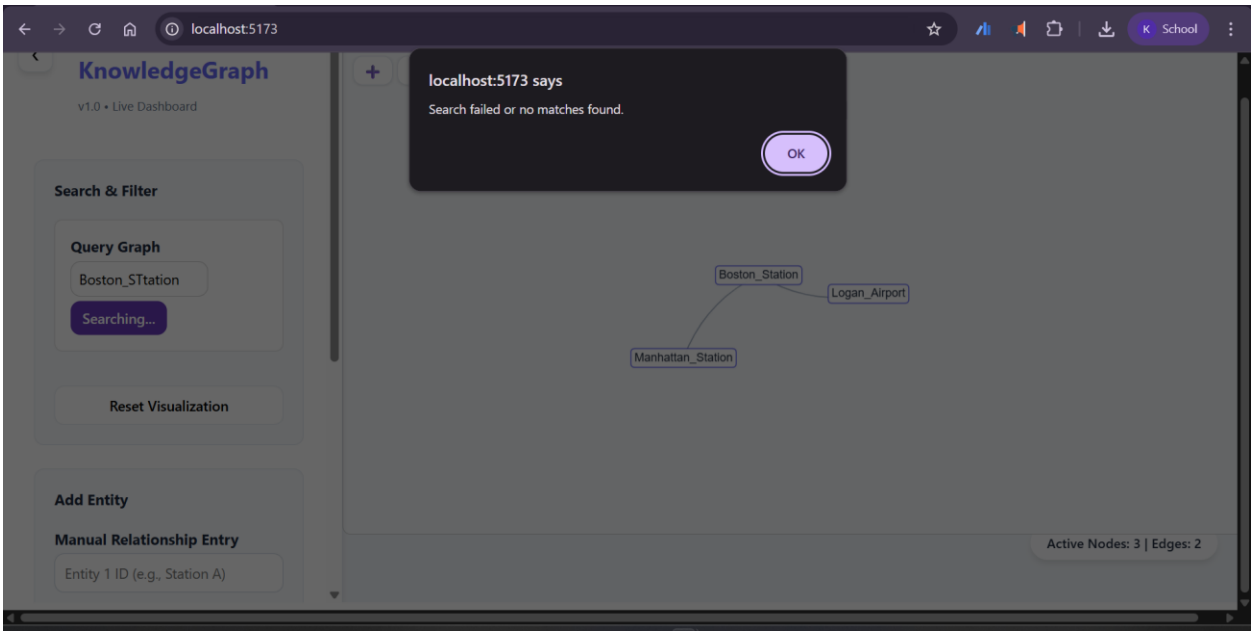

Figure 11 Bulk Upload - Transportation Use Case - Query – Failure Scenario